Efficient LLM-based Subgraph Retrieval for Multi-Hop Knowledge Base Question Answering

Duanyang Yuan, Sihang Zhou*, Xiaoshu Chen, Haoyuan Chen, Ke Liang, Jian Huang*

***Abstract*—Multi-hop Knowledge Base Question Answering (KBQA) aims to find answer entities in the knowledge base that are multiple hops away from the entities in the question. Information retrieval-based (IR-based) methods extract a pivotal subgraph from the entire KB to locate candidate answers and then evaluate their plausibility through semantic matching with the question. However, we observed that the extracted subgraphs often include nodes that are weakly related or irrelevant to the question. Without a proper node filtering mechanism, the number of irrelevant nodes grows as the number of hops increases, leading to excessive consumption of computational resources. To address these challenges, this study introduces an efficient LLM-based subgraph retrieval method for multi-hop knowledge base question answering, M-ER. The framework leverages Monte Carlo Tree Search (MCTS) to transform subgraph exploration into a tree-structured search process. During the MCTS selection phase, nodes that are highly relevant to the question are prioritized for inclusion in the subgraph, eliminating the need to traverse all nodes in the KB. The framework further incorporates a large language model (LLM) to refine the search direction, ensuring that exploration remains focused on nodes relevant to the question. In addition, selected nodes are quantitatively scored, and these scores are fed back into the node selection process to effectively filter out irrelevant candidates, thereby improving the quality of the subgraph. This mechanism not only narrows the search space but also enhances the overall efficiency of multi-hop KBQA. Experiments on the WebQSP benchmark demonstrate that M-ER achieves 78.88% on the Hits@1 metric, while also improving computational efficiency. These results not only validate the effectiveness of M-ER, but also offer a viable technical path to balance performance and computational efficiency[1].**

***Index Terms*—Multi-hop Question Answering, MCTS, Knowledge Base, Large Language Model.**

I. Introduction

KNOWLEDGE base question answering (KBQA) [1] aims to retrieve answers from structured knowledge bases (e.g., Freebase [2], DBpedia [3]) in response to natural language queries. Current methodologies can be divided into two paradigms: semantic parsing-based (SP-based) [4]–[7] and information retrieval-based (IR-based) [8], [9]. SP-based approaches translate questions into executable logical forms [6], [10] but require expensive annotation efforts. In contrast, IR-based methods directly represent and rank candidate entities based on their relevance to the input questions.

Extracting a question-relevant subgraph and reasoning within it is a key branch of IR-based methods. The performance of subgraph extraction is mainly influenced by two factors: subgraph construction quality and answer coverage rate. Subgraph construction directly affects the efficiency and accuracy of the reasoning process, as reasoning paths are formed by expanding nodes and relations within the subgraph. The answer coverage rate defines the upper bound of the performance [11]. Current subgraph extraction methods can be categorized into two main approaches: the k-hop neighbor method and the scoring-based method. The k-hop neighbor approach constructs subgraphs by retrieving a fixed number of hops around the topic entity without requiring additional search heuristics. However, KBQA is sensitive to subgraph size, since a small subgraph may exclude correct answers, while a large one may introduce excessive noise. Scoring-based methods perform KBQA by quantifying the semantic association between the question and KB elements. NSM [12] adopts a teacher-student framework with bidirectional reasoning to generate intermediate supervision signals for answer extraction. SR [13] introduces a trainable subgraph retriever decoupled from inference, and generates accurate question-related subgraphs through weakly supervised pre-training and end-to-end fine-tuning. When combined with NSM [12], this approach sets a new performance benchmark.

Existing scoring-based methods typically construct subgraphs by computing semantic similarity between the question and knowledge base elements (entities or relations), as shown in Figure 1(a). Representative methods such as SR [13] use language models to compute semantic similarity. However, when a single entity has relations corresponding to diverse attributes, these methods may retain relations that are weakly related or irrelevant to the question, expanding the search space and reducing answer accuracy. For example, for the question *What language did Jewish people speak?*, we analyze the total number of one-hop neighboring relations of the topic entity *Jewish people*, along with the names and frequencies of the neighboring relations filtered based on similarity, as shown in Table I. From this, three issues can be observed: (1) The topic entity *Jewish people* has rich attribute information, linking to language-related entities via the relation *people.ethnicity.languages_spoken* and to person-related entities via the relation *people.ethnicity.people*. All these links originate from a single entity. (2) Existing methods typically compute similarity between the query [*What*

Duanyang Yuan, Sihang Zhou, Haoyuan Chen, and Jian Huang are with the College of Intelligence Science and Technology, National University of Defense Technology, Changsha 410073, China, and also with the National Key Laboratory of Equipment State Sensing and Smart Support, Changsha 410073, China. (e-mail: {shzhou, huang_jian}@nudt.edu.cn)

Xiaoshu Chen and Ke Liang are with the College of Computer Science and Technology, National University of Defense Technology, Changsha 410073, China

∗ Corresponding Authors

[1]https://github.com/DuanyangYuan/M-ER

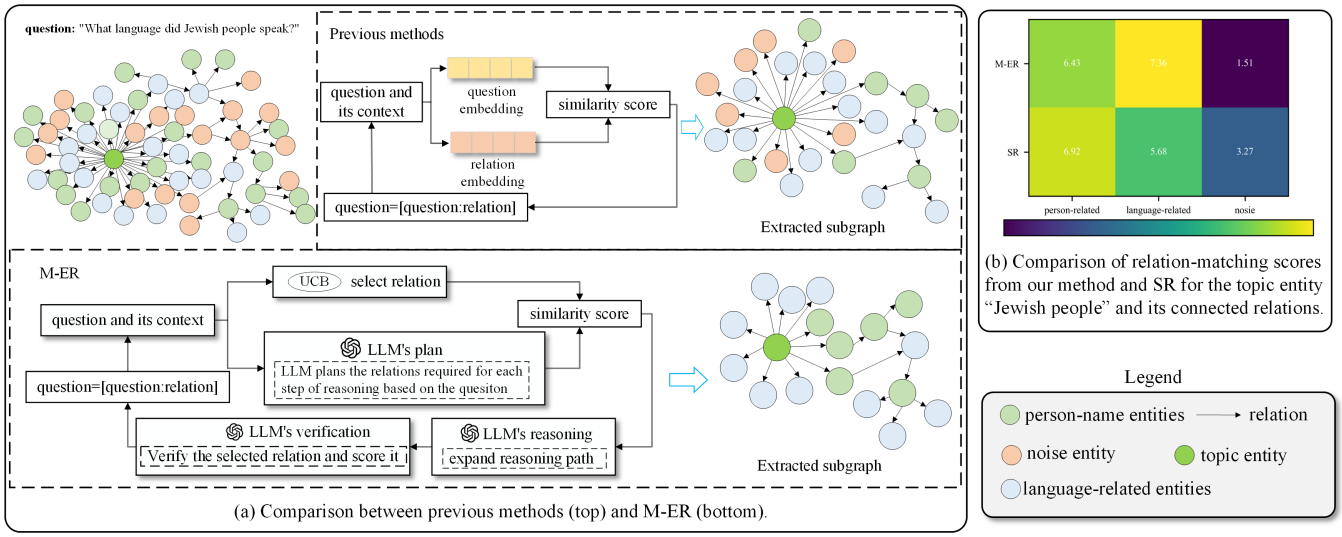


Fig. 1. Illustrates the challenges faced by previous methods (SR) and how M-ER alleviates them. (a) A comparison between SR (top) and M-ER (bottom) is presented. SR expands subgraphs primarily based on semantic similarity, which tends to introduce nodes that are weakly related or irrelevant to the question. These irrelevant expansions distract the reasoning process and incur unnecessary computational overhead. In contrast, M-ER incorporates LLM-based reflection, reasoning, and evaluation stages. M-ER evaluates each candidate node for expansion by considering both the current subgraph context and the question, thereby explicitly estimating its potential contribution to answering the question. (b) Comparison of relation-matching scores from our method (M-ER) and SR. Specifically, M-ER computes similarity between the predicted relation and each relation linked to the topic entity, and SR measures similarity between the query and those relations. While SR struggles to distinguish answer-relevant relations from those that are weakly related or irrelevant to the question, M-ER effectively prioritizes relations that are answer-relevant (e.g., *language-related*) and suppresses weakly relevant relations (e.g., *person-related*).

TABLE I

THE ONE-HOP NEIGHBORS OF THE TOPIC ENTITY *Jewish people* ARE SELECTED BY CALCULATING SIMILARITY SCORES BETWEEN THE QUESTION AND THE RELATIONS. ONLY THOSE NEIGHBORS THAT MEET THE PREDEFINED THRESHOLD CAN BE ADDED TO THE SUBGRAPH. THE TERM "FREQUENCY" INDICATES HOW OFTEN THE ONE-HOP RELATIONS APPEARS IN THE DATASET.

Question: What language did Jewish people speak?
Topic entity: Jewish people
The number of one-hop neighbor relations of the topic entity: 5308

Relation Name	Frequency	Relation Name	Frequency
people.ethnicity.people	5149	people.ethnicity.includes_groups	13
common.topic.webpage	47	book.book_subject.works	12
people.ethnicity.languages_spoken	25	military.military_combatant.military_conflicts	4
fictional_universe.ethnicity_in_fiction.characters_of_this_ethnicity	25	film.film_subject.films	3
type.object.type	16	common.topic.image	2

language did Jewish people speak?; Jewish people] and all relations connected to the topic entity *Jewish people*. As a result, the person-related relation *people.ethnicity.people* may be retained during retrieval even though only language-related information is relevant to answering the question. Notably, person-related relations (e.g., *people.ethnicity.people*) account for 97%, far exceeding relevant language-related relations (e.g., *people.ethnicity.languages_spoken*), thereby overshadowing crucial language-related links and propagating semantic noise to subsequent hops. This propagation can lead to a combinatorial explosion, reducing overall performance. (3) The topic entity *Jewish people* has 5,308 one-hop neighboring relations. Extending the subgraph based on this introduces substantial computational overhead.

To address these challenges, Coke [14] formulates reasoning as a trade-off between accuracy and computational cost, aiming to achieve cost-effective inference. Although effective in principle, this approach relies on global answer matching over the entire KB, which introduces substantial noise and increases the overall computational burden. MCTS provides a more targeted exploration strategy [15]–[17]. By simulating future states through random sampling, it incrementally constructs a search tree where the nodes correspond to possible reasoning steps within the subgraph. This strategy focuses computation on the most promising nodes, reducing the need for exhaustive enumeration. By balancing exploration and exploitation, MCTS narrows the scope of the search and improves computational efficiency [17]–[20]. Extending this idea, KBQA-o1 [21] leverages MCTS to balance exploration quality and search range, while also adopting incremental fine-tuning to reduce reliance on labeled data. Its limitation lies in disregarding the structural and semantic context of the knowledge graph, making it vulnerable to cascading errors from incorrect intermediate predictions. MCTS-KBQA [22] introduces a task-specific scoring function to improve answer selection. However, it lacks explicit guidance for selecting subsequent search directions and does not fully exploit the structural information within subgraphs. Consequently, it often terminates at intermediate states and treats them as final answers, leading to lower accuracy and unstable performance.

In addition, MCTS often struggles in scenarios where a single entity is connected to relations corresponding to diverse attributes, as weakly related or irrelevant nodes may be retained during exploration. This reduces the model's ability to discriminate answer-relevant semantics, leading to the inclusion of noisy entities and a degradation in overall performance. Therefore, M-ER is designed to incorporate MCTS into KBQA while explicitly filtering irrelevant relations.

Large language models (LLMs), on the other hand, exhibit strong contextual reasoning capabilities [23], [24] and can guide node selection toward answer-relevant directions [18], [25]. Therefore, this study proposes an efficient LLM-based subgraph retrieval method for multi-hop knowledge base question answering (M-ER), which integrates the semantic reasoning capabilities of LLMs with the global search efficiency of MCTS, as shown in Figure 1(a). This approach jointly activates model capabilities from two perspectives: internal (semantic reasoning within the model) and external (exploration of the KB). Specifically, LLMs interpret the question to predict the next search direction, which is then matched with candidate nodes based on semantic similarity. Meanwhile, selected nodes are evaluated based on their contribution to the final answer, and these scores are used to refine the reflection process, thus improving the alignment between the model's internal knowledge and external information. It can be seen from Figure 1 (b) that this method can effectively reduce the similarity scores of irrelevant nodes, thus improving the quality of the subgraph. The main contributions of this work are in three aspects:

- We combine MCTS and LLM for subgraph construction and solve the trade-off between semantic focus and search efficiency by balancing exploration and exploitation. Experimental results on WebQSP demonstrate that M-ER achieves 78.88% on the Hits@1 metric, surpassing the previous relevance-driven method (SR) by 7.08%.
- We design a guidance mechanism where LLMs steer MCTS toward answer-relevant subgraph expansions, effectively mitigating semantic drift.
- We develop a subgraph evaluation mechanism that assigns quantitative scores to selected nodes, enabling feedback-driven optimization of reasoning paths within the subgraph. This mechanism opens new avenues for KBQA.

II. Related Work

A. Subgraph Retrieval-based KBQA

We review subgraph retrieval methods in KBQA from two aspects: the k-hop neighbor method and the scoring-based method.

1) K-hop neighbor methods: The k-hop neighbor method can extract subgraphs in two ways. The k-hop neighbors of the topic entity are selected directly as a subgraph. The other method (e.g., RASR [11]) first computes similarity between the question and all relations. It then selects the most relevant relations to construct the initial 1-hop subgraph, which is further expanded to form the 2-hop subgraph. However, this rigid hop-based selection may exclude correct answers when the hop size is too small, while introducing substantial noise when the hop size is too large.

2) Scoring-based methods: The methods perform KBQA by quantifying the semantic association between the question and the KB elements. The key is to design a similarity scoring function to screen the key information. The early key-value memory network [26] stores context as key–value pairs, enabling the model to learn how keys can be used to retrieve values relevant to the question. However, this method is limited by hardware memory. With the development of end-to-end neural networks, GRAFT-Net [27] retrieves the nodes and relations related to the question in the KB to construct a subgraph containing high-probability answers. Node representations are then learned over the constructed subgraph, based on which answer candidates are inferred. This method lacks supervision and thus cannot constrain the reasoning process. To address this limitation, some studies propose improvements, such as PullNet [5], SR [13], FC-KBQA [28], CBR-SUBG [29], and Query Graph Ranking [30]. However, these methods primarily focus on the similarity between the question and the KB, without considering whether the retrieved nodes actually contribute to answering the questions. As a result, irrelevant information that is similar to the question is introduced, leading to subgraph redundancy and unnecessary consumption of computational resources.

B. LLM-based KBQA

The advent of LLMs has greatly improved the reliability of KBQA performance [31], [32]. The LLM-based KBQA methods can generally be categorized into several types. The first category adopts the generation-retrieval paradigm, in which LLM generates the initial answer or logical rules, and then refines it using entity/relation retrieval or document verification (e.g., ChatKBQA [33], FuSIC-KBQA [34], GenGround [35], RoG [36]). ChatKBQA [33] employs an innovative generate-then-retrieve approach. A fine-tuned LLM first produces a skeleton of a logical form, which is then aligned with the knowledge graph via unsupervised retrieval of entities and relations. RoG [36], on the other hand, integrates structural knowledge from KGs through its planning–retrieval–reasoning framework, generating KG-grounded relation paths as faithful plans for interpretable reasoning. Although these methods reduce dependence on explicit semantic parsers, they remain sensitive to retrieval noise and incur high computational costs.

The second category focuses on question decomposition and step-by-step reasoning (e.g. ICSU [37], GRV-KBQA [38]), thereby enhancing interpretability. GRV-KBQA [38] adopts a three-stage framework that decouples logical structure generation, semantic grounding, and structure-aware verification. It first decomposes a complex query into a chain of subquery templates and then fills these templates to produce executable logical forms for question answering. However, such decomposition methods are limited when relevant information exceeds the predefined query scope. Other methods, such as RGR-KBQA [39], follow a "retrieve–generate–retrieve" three-stage framework. In this approach, cosine similarity is used to extract the most relevant relation chains, which enhances

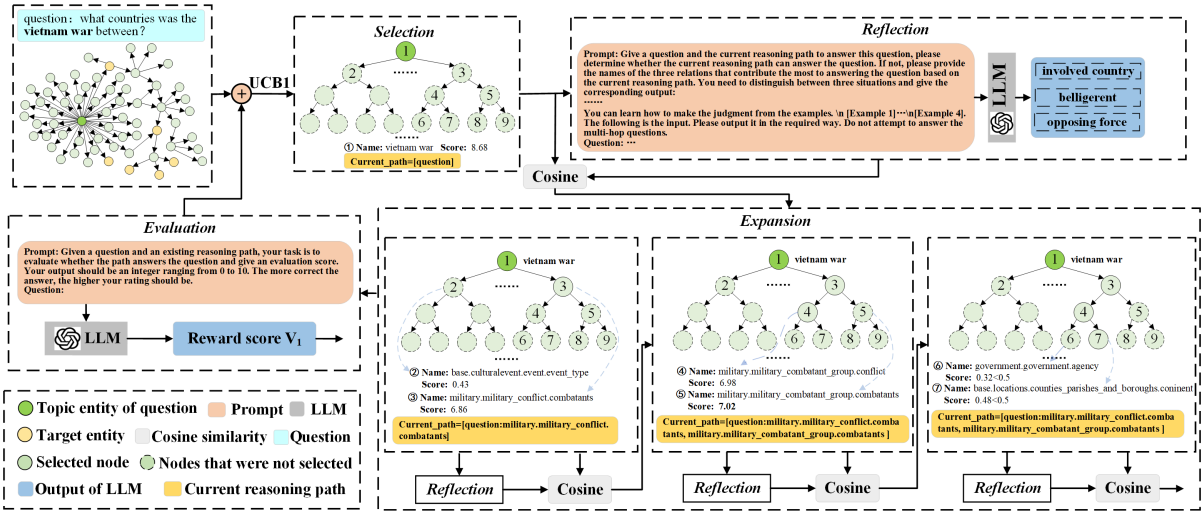


Fig. 2. The framework of M-ER. During the selection phase, the LLM-guided reward score optimizes the node exploration strategy, aiming to select the node most relevant to answering the question while minimizing the inclusion of irrelevant nodes. In the reflection stage, the LLM assesses the reasoning paths within the current subgraph and suggests several candidate nodes for the next search direction. The expansion phase leverages both the LLM-guided reflection and the existing KB to collaboratively determine the next search direction. This process enables the generation of multiple reasoning paths, which go beyond the constraints of manual rules. In the evaluation phase, the nodes identified in the expansion phase are scored. The process is simulated iteratively, with each cycle updating the reward score recursively to the UCB1 value of the parent node.

semantic understanding and improves generation accuracy. Nonetheless, most of these methods are heavily dependent on similarity metrics and prompt engineering, which introduces substantial noise and hampers overall performance. Currently, LLM-based KBQA focuses mainly on how to make full use of the inherent knowledge and the strong reasoning ability of LLM in a large KB, while ignoring the role of subgraphs and efficient search strategies. Therefore, our work is dedicated to bridging these knowledge gaps.

III. Method

Extracting and reasoning over a subgraph that is relevant to the question is a typical approach in KBQA. To extract valuable information from the graph and minimize information loss, we propose an efficient LLM-based subgraph retrieval method for multi-hop knowledge base question answering, called M-ER. The overall framework is illustrated in Figure 2. In this process, the subgraph not only provides the information required to answer the question but also captures the reasoning paths connecting the topic entity to the answer node. Each iteration builds on this subgraph to produce a specific reasoning path. By exploring these expanding subgraphs through a tree structure, the LLM can introduce sufficient diversity, thereby fully activating its reasoning capabilities.

A. Predefined Relations in Subgraph

As shown in Figure 3, before the transformation, e_1, e_2, e_3, and e_4 denote neighboring entity nodes, while r_1, r_2, r_3, and r_4 denote relation edges connecting the topic entity to its neighbors. After the transformation, relation edges are reified as relation nodes. Consequently, r_1, r_2, r_3, and r_4 become explicit nodes in the subgraph. Transforming relation edges into relation nodes provides significant advantages for subgraph reasoning tasks [40]. By reifying relations as

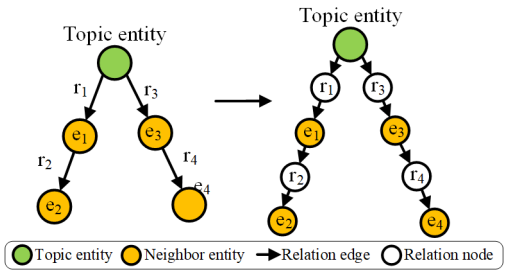


Fig. 3. Transforming relation edges into relation nodes. In the left subfigure, e_1, e_2, e_3, and e_4 denote entity nodes. After transformation, as shown in the right subfigure, r_1, r_2, r_3, and r_4 denote relation nodes. Specifically, we regard the topic entity node as the 0-hop node.

nodes, the graph explicitly encodes second-order semantic associations through direct topological connections between relation nodes. As illustrated in Table I, when handling queries such as *What language did Jewish people speak?*, the relation-node formulation enables direct similarity computation between the question semantics and relation nodes (e.g., *people.ethnicity.languages_spoken*). This design reduces the semantic deviation introduced by traditional methods that require indirect matching through entity intermediaries.

B. Selection Based on Reward Score

The process begins by selecting a node $Leaf_i$ from the tree hierarchy. In each node selection, the candidate nodes are evaluated using the Upper Confidence Bound (UCB1) metric [41], which integrates their reward scores to estimate their contribution to answering the question, as shown in equation 1. A greedy algorithm is employed, selecting the node with the highest score at each step. Specifically, the root node is initialized with the topic entity of the question, denoted as $Q_{original}$. When multiple nodes produce equivalent UCB1

scores, a randomized selection protocol is activated to preserve exploration diversity. This selection process recursively explores the branching structure until it reaches a terminal leaf node, thereby determining the search trajectory.

$$UCB1_i = V_i + c \cdot \sqrt{\frac{2 \cdot \ln N}{n_i}} \quad , \tag{1}$$

where V_i denotes the empirical average reward score of node $Leaf_i$, as generated by the LLM during the evaluation phase; c is the exploration constant, set to 0.5 [17], [42]; n_i represents the visit count of this node, and N is the visit count of its parent node.

C. Reflection and Evaluation Based on LLM

Reflection. In this phase, a reflection mechanism [43] is employed. The LLM first generates a brief reflection based on the current path C_i^{path} to propose candidate next-hop nodes, denoted as $Leaf_{LLM1}$. Subsequently, $Leaf_{LLM1}$ is processed through equation 4 and 5 to determine the actual next-hop node $Leaf_{i+1}$. Finally, this reflection assesses whether appending $Leaf_{i+1}$ to C_i^{path} is sufficient for answering the question. If so, the model outputs an <end> flag to terminate the path generation process. Otherwise, it continues by generating the next-hop node $Leaf_{LLM2}$. Through this iterative process, the LLM updates the reasoning path C_{i+1}^{path}, formed by concatenating $Leaf_{i+1}$ into C_i^{path}.

$$C_{i+1}^{path} = [Q_{original}; Leaf_0; Leaf_1; ...; Leaf_i; Leaf_{i+1}]. \tag{2}$$

This evolving path C_{i+1}^{path} serves as the context for subsequent LLM reasoning, guiding generation by reflecting prior steps and dynamically adjusting output diversity.

Evaluation The LLM assigns reward scores V_{i+1} to newly generated nodes $Leaf_{i+1}$ based on three criteria: single-step correctness, semantic coherence, and node validity [25]. Specifically, when erroneous steps emerge during intermediate stages, the corresponding node $Leaf_{i+1}$ receives a reduced reward score. This penalization mechanism discourages the selection of such branches, mitigating robustness degradation caused by mid-process errors. The scoring is formalized as follows:

$$V_{i+1} = \text{LLM}(question : C_{i+1}^{path}, prompt), \tag{3}$$

where C_{i+1}^{path} denotes the accumulated path comprising all selected nodes up to the $i+1$-th iteration, and *prompt* indicates the system prompt used to guide the LLM's evaluation.

D. Subgraph Expansion Based on Constraints

During the previous phase, although the LLM generates new nodes $Leaf_{LLM1}$, directly using them as input often results in unstable performance. Therefore, node expansion is constrained by the following formulas.

$$CosHop_i = cos(Leaf_{LLM1}, LeafHop_i), \tag{4}$$

$$Leaf_{i+1} = \{\, v \in LeafHop_i \mid CosHop_i(v) > \textit{threshold} \,\}, \tag{5}$$

where $LeafHop_i$ denotes the set of one-hop nodes of $Leaf_i$ in the KG; $CosHop_i$ represents the cosine similarity between each node in $LeafHop_i$ and node $Leaf_{LLM1}$. $Leaf_{i+1}$ refers to the subset of $LeafHop_i$ whose cosine similarity $CosHop_i$ exceeds the predefined threshold. The *threshold* is set to 0.5.

E. Reasoning Path Simulation

Building upon the selection and expansion mechanisms, this stage uses a multi-criteria termination protocol to enhance exploration efficiency and maintain semantic integrity. This process maintains a path state tuple (C_i^{path}, V_i, n_i) and updates it during each simulation, where $V_i = \{V_0, ..., V_i\}$ contains historical reward scores for the current path C_i^{path}, and n_i records the visit count of the current node. Path simulation terminates when any of these conditions are met:

- *Path Repetition*: The simulation terminates if the newly generated path is identical to the previous one.
- *Confidence Collapse*: The average of the evaluation scores falls below the threshold:

$$P_{end} = Cos(C_i^{path}, Q_{original}) < \gamma_{score}. \tag{6}$$

- *Structural Termination*: The expansion terminates when a leaf node is detected.
- *Solution Verification*: The LLM outputs an <end> flag after semantic validation.

The simulation integrates structured KG semantics through dual-channel processing: explicit path sequences C_i^{path} guide the LLM's reasoning, while implicit embedding similarity $CosHop_i$ ensures topological consistency. This method balances reasoning patterns with graph constraints, addressing both hallucination issues and computational efficiency requirements.

In the above process, we integrate the question and the existing path by serializing them. This serialization enables the reasoning path selected by MCTS to provide structured inference steps for the LLM. Meanwhile, it provides semantic support for subgraph extraction, ensuring that each inference step aligns with the actual relations in the KG.

F. Backpropagation of Reward Score

Backpropagation refines the selection phase by recursively propagating reward increments from leaf nodes to parent nodes through a bottom-up feedback mechanism. This upward propagation dynamically adjusts path weights in probabilistic models. The mathematical formulation is as follows:

$$r_{N_{increment}} = \frac{\sum_{i=1}^{\text{branch}} Visits_{N_{children}} \cdot r_{N_{children}}}{\sum_{i=1}^{\text{branch}} Visits_{N_{children}}}, \tag{7}$$

$$Visits_{N_{children}} = Visits_{N_{children}} + 1, \tag{8}$$

$$r_{parent} \leftarrow \alpha \cdot r_{parent} + (1 - \alpha) \cdot r_{N_{children}}, \tag{9}$$

TABLE II
STATISTICS OF THE DATASETS USED IN THIS PAPER.

Dataset	KG	#Train	#Validation	#Test
WebQSP	Freebase	2,848	250	1,639
CWQ	Freebase	27,639	3,519	3,531
MetaQA 1-hop	WikiMovies	96,106	9,992	9,947
MetaQA 2-hop	WikiMovies	118,980	14,872	14,872
MetaQA 3-hop	WikiMovies	114,196	14,274	14,274

where $Visits_{N_{children}}$ denotes the number of visits to node $N_{children}$, which is used to balance the contribution of the frequently visited node to the increment. $r_{N_{increment}}$ denotes the increment in the parent node's reward provided by the child node, and α is used to balance the original reward of the parent node and the increment from the child node. We set α to 0.7. These phases are repeated until the termination condition is met or the maximum tree depth is reached.

IV. EXPERIMENTS

In this phase, we conduct the experiments to address the following research questions: (1) Does our method improve the final QA performance? (2) Does our subgraph retrieval method generate smaller and higher-quality subgraphs? (3) Is each of the intermediate steps in the method effective?

A. Experimental Settings

1) Datasets: We evaluate our method on two KBQA datasets based on Freebase and one dataset based on the WikiMovies domain. A summary of dataset statistics is provided in Table II. Freebase is a large open source KB composed of structured data, comprising approximately 68 million entities and 1 billion relations. WebQuestionsSP (WebQSP) [44], derived from the WebQuestions dataset [45], augments natural language questions with SPARQL annotations and evaluates answers by executing the corresponding queries on Freebase. This enables a more rigorous assessment of whether a question can be answered. It contains 4,737 answerable natural language questions and splits the train/dev/test sets as 2,848/250/1,639. Complex WebQuestions 1.1 (CWQ) [46], based on WebQSP, introduces questions with complex structures such as conjunctions, superlatives, comparatives, and compositions. The dataset consists of 34,689 cases, including 27,639 training instances, 3,531 test instances, and 3,519 validation cases. MetaQA [47] is a large-scale multi-hop QA dataset constructed from a movie-domain knowledge graph. It includes over 400k questions across 1-hop, 2-hop, and 3-hop settings. Across all three subsets, MetaQA presents a high proportion of complex multi-hop queries, making it a suitable testbed for assessing model robustness beyond Freebase.

2) Evaluation Metrics: In QA tasks, our methodology employs two complementary metrics to assess model performance. Since a question may have multiple correct answers, we use Hits@k to evaluate retrieval performance. Hits@k measures the percentage of questions for which at least one correct answer appears in the top-k retrieved answers. This serves as an upper bound indicator of the model's ability to retrieve relevant answers within limited search conditions. To complement this retrieval-focused evaluation, the F1-score is used to capture the trade-off between precision and recall in answer selection.

The evaluation framework also incorporates a cost-effectiveness analysis [48], assessing the trade-off between generation quality and computational cost. Generation quality is measured using task-specific metrics, such as Hits@1, which evaluates the model's precision in ranking the most relevant answer at the top. Computational cost is measured by the overall runtime of our method. Cost-effectiveness is then quantified as the ratio of Hits@1 to overall runtime, enabling direct comparison of how different models balance predictive accuracy against computational demands.

3) Implementation Details: For our method, the reasoning depth is set to 3 for both the WebQSP and MetaQA datasets, and to 4 for the CWQ dataset. The number of MCTS simulations is set to 5. For baseline methods, we employ DeepSeek-V3 as the LLM backbone due to its strong QA performance and relatively low computational cost [49], [50]. In addition, all experiments are conducted on 2 NVIDIA RTX 4090 24GB GPUs.

4) Baseline Models: We evaluate our model against representative baselines from three categories. For IR-based models, we consider Key-Value Memory Network (KV-MemNN) [26], EmbedKGQA [51], GRAFT-Net [27], NSM [12], PullNet [5], NSM_{+p} [12], and SR [13]. For SP-based models, we include SPARQA [6], TextRay [52], MULTIQUE [53], SRGCN [54], and Relational Chain-based Embedded KGQA (Rce-KGQA) [55]. For LLM-based methods, we compare with ToG [56], GoG [57], StructGPT [58], ChatGPT [36], Flan-T5-XL [59], FlexKBQA [60], KBQA-o1 [21], and MCTS-KBQA [22].

B. Overall QA Evaluation

We compare the performance of M-ER against previous state-of-the-art methods. Table III reports the results on the WebQSP and CWQ datasets, while Table IV reports the performance on the MetaQA dataset.

For the WebQSP and CWQ datasets, the results in Table III demonstrate that our method achieves strong performance across both benchmarks. On the WebQSP dataset, our framework obtains 78.88% Hits@1 and 72.17 F1-score, significantly outperforming all baseline models. Compared to the SP-based method, Rce-KGQA, our approach improves Hits@1 by 8.48%. On the CWQ dataset, M-ER achieves 73.46% Hits@1 and 65.28 F1-score, surpassing IR-based and SP-based methods. For example, compared to the best IR-based model, SR, our method yields gains of 20.66% in Hits@1 and 19.48 in F1-score on the CWQ dataset. While LLM-based approaches, such as GoG, ToG, StructGPT, and MCTS-KBQA, show competitive performance, our method still achieves the best results. Notably, it surpasses GoG by 2.68% in Hits@1 on the WebQSP dataset, and by 19.16% in Hits@1 on the CWQ dataset. Similarly, compared with MCTS-KBQA, the Hits@1 metric increases by 4.28% and 5.16% on the two datasets, respectively. These consistent improvements highlight the robustness and generalization of our framework across different datasets.

TABLE III
OVERALL QA EVALUATION OF WEBQSP AND CWQ DATASETS. HITS@1 (%) AND F1-SCORE ARE REPORTED. ∗ REFERS TO THE REPRODUCED RESULT FROM OUR WORK. **BOLD** NUMBERS DENOTE THE BEST PERFORMANCE, AND UNDERLINED NUMBERS DENOTE THE SECOND-BEST PERFORMANCE.

Models	WebQSP		CWQ	
	Hits@1	F1-score	Hits@1	F1-score
SP-based models				
SPARQA [6]	-	-	31.57	-
TextRay [52]	72.16	60.3	40.83	33.87
MULTIQUE [53]	69.84	61.16	37.61	31.24
SRGCN [54]	-	56.6	-	-
Rce-KGQA [55]	70.4	-	48.3	-
IR-based models				
KV-Mem [26]	46.6	34.5	18.4	15.7
EmbedKGQA [51]	66.6	-	32.0	-
GRAFT-Net [27]	68.7	62.3	36.8	32.7
NSM* [12]	67.1	63.4	47.2	41.9
PullNet [5]	68.1	-	45.9	-
NSM_{+p} [12]	71.5	64.2	47.9	42.4
SR* [13]	71.8	65.4	52.8	45.8
LLM-based				
GoG* [57]	76.2	66.5	54.3	50.1
ToG* [56]	77.2	70.9	49.8	46.5
StructGPT* [58]	73.3	69.5	-	-
ChatGPT [36]	66.8	-	39.9	-
Flan-T5-XL [59]	31.0	-	14.7	-
FlexKBQA [60]	-	60.6	-	-
KBQA-o1 [21]	-	67.0	-	-
MCTS-KBQA* [22]	74.6	65.4	68.3	64.2
Our method	**78.88**	**72.17**	**73.46**	**65.28**

TABLE IV
COMPARISON OF HITS@1 (%) ACROSS STATE-OF-THE-ART METHODS ON THE METAQA DATASET. ∗ INDICATES THAT THE RESULTS OF THE CORRESPONDING METHOD ARE REPRODUCED BY OUR WORK UNDER THE SAME EXPERIMENTAL SETTINGS. THE BEST PERFORMANCE IN EACH COLUMN IS INDICATED IN **BOLD**, WHILE THE SECOND-BEST PERFORMANCE IS UNDERLINED.

Models	MetaQA 1-hop	MetaQA 2-hop	MetaQA 3-hop	Average
Rce-KGQA [55]	98.3	99.7	97.9	98.6
KV-Mem [26]	96.2	82.7	48.9	75.9
EmbedKGQA [51]	97.5	98.8	94.8	97.0
GRAFT-Net [27]	97.0	94.8	77.7	89.8
NSM* [12]	97.3	99.6	98.2	98.4
PullNet [5]	97.0	**99.9**	91.4	96.1
NSM_{+p} [12]	97.3	**99.9**	98.9	98.7
StructGPT* [58]	94.7	94.3	79.6	89.5
Our method	**98.6**	99.7	**99.3**	**99.2**

For the MetaQA dataset, M-ER achieves the highest average performance, with an average Hits@1 score of 99.2%, surpassing all other methods. Specifically, M-ER obtains Hits@1 scores of 98.6%, 99.7%, and 99.3% for 1-hop, 2-hop, and 3-hop datasets, respectively. It outperforms PullNet on the 1-hop (97.0%) and 3-hop (91.4%) datasets, while slightly underperforming compared to PullNet and NSM_{+p} on the 2-hop dataset (99.9%). These results highlight M-ER's ability to generalize across knowledge graphs from different domains, suggesting that the observed performance gains are not limited to Freebase.

Summary Our M-ER achieves superior performance on the WebQSP, CWQ, and MetaQA datasets, with significant advantages in multi-hop KBQA. These improvements stem from the integration of a key node selection mechanism that focuses on answer relevance.

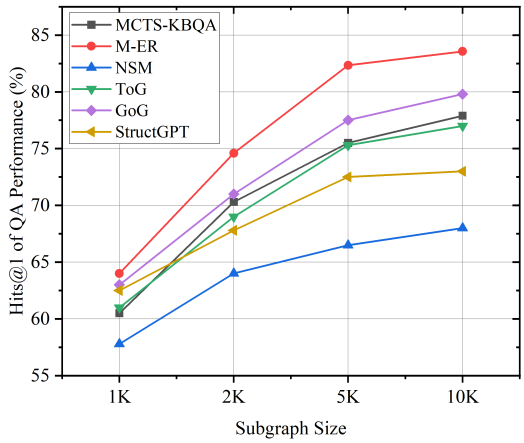


Fig. 4. The impact of different subgraph sizes on the WebQSP dataset for the Hits@1 metric. The baselines for comparison are MCTS-KBQA, NSM, ToG, GoG, StructGPT, and M-ER.

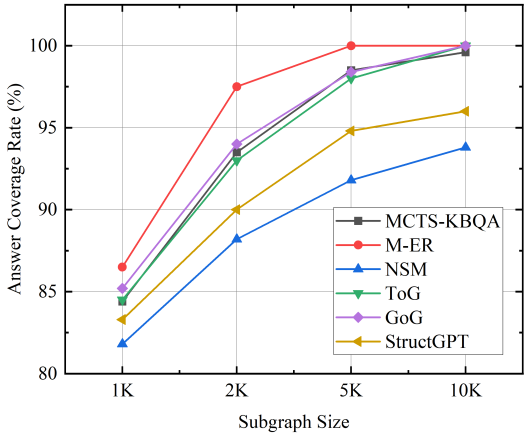


Fig. 5. The impact of different subgraph sizes on the WebQSP dataset for the answer coverage rate. The baselines for comparison are MCTS-KBQA, NSM, ToG, GoG, StructGPT, and M-ER.

C. Evaluation of Retrieved Subgraph

As shown in Figure 4, the experimental results clearly demonstrate the effectiveness of M-ER in terms of the Hits@1 metric across different subgraph sizes on the WebQSP dataset. M-ER demonstrates a more pronounced performance improvement as the subgraph size increases, rapidly surpassing all baseline methods, including MCTS-KBQA [22], NSM [12], ToG [56], GoG [57], and StructGPT [58]. Notably, M-ER achieves over 80% Hits@1 even with a subgraph size of only 5K, showcasing its capability to identify high-quality answers within smaller search spaces.

Figure 5, which presents the answer coverage rate across different subgraph sizes, further validates the effectiveness of M-ER. It consistently outperforms all baselines, reaching near-perfect coverage at 5K and maintaining 100% coverage at 10K, while other models require larger subgraphs to achieve compa-

rable performance. These results underscore the robustness of M-ER in identifying correct reasoning paths while preserving essential semantic and structural information during subgraph pruning.

These empirical trends collectively reflect the superior scalability and adaptability of the M-ER framework. Compared with conventional models, M-ER benefits from its hybrid architecture that integrates MCTS with a node scoring mechanism. This synergy allows the model to effectively suppress irrelevant branches while favoring nodes that are semantically rich and relevant to the question. As a result, M-ER scales to larger graphs and more complex queries, making it particularly well-suited for real-world QA over large-scale knowledge graphs.

Summary Our method significantly outperforms MCTS-KBQA, NSM, ToG, and other baselines in both Hits@1 and answer coverage rate, achieving superior performance with smaller subgraph sizes. These results demonstrate the effectiveness and efficiency of M-ER in enhancing QA performance while reducing computational overhead.

D. Cost-Effectiveness Analysis

The resource consumption of our method is primarily influenced by two factors: the use of LLMs and the design of the MCTS strategy. To evaluate the impact of these two components on computational resources, we design three experiments on the WebQSP dataset and report the wall-clock GPU hours, overall runtime, input/output token counts, and cost-effectiveness. The three experimental settings are summarized as follows.

- **Non-LLM Setting.** When LLMs are not involved, this experiment evaluates the wall-clock GPU hours, overall runtime, cost-effectiveness, and Hits@1 of M-ER (w/o LLM) by comparing it with classic IR-based KBQA methods, including SR [13], NSM [12], and DPR [61] + Reranker [62], as shown in Table V.
- **LLM-based Setting.** When LLMs are involved, this experiment evaluates overall runtime, input/output token counts, cost-effectiveness, and Hits@1 of M-ER (w/ LLM) by comparing it with SOTA LLM-driven KBQA methods, such as ToG [56], GoG [57], and MCTS-KBQA [22], as shown in Table VI.
- **Search Strategy Comparison Setting.** This experiment evaluates overall runtime, input/output token counts, cost-effectiveness, and Hits@1 by analyzing the impact of different search algorithms. Specifically, M-ER replaces MCTS with alternative search strategies, including ReAct [63], depth-first search (DFS) [64], and breadth-first search (BFS) [65], as shown in Table VII.

In the first set of experiments (Non-LLM Setting), we evaluate the wall-clock GPU hours, overall runtime, cost-effectiveness, and Hits@1 of different methods, as shown in Table V. The results indicate that M-ER (w/o LLM) achieves the best performance across all metrics compared to the baselines. Specifically, M-ER attains the highest Hits@1 (72.37%) and cost-effectiveness (44.13), while also exhibiting the lowest wall-clock GPU time (0.74s) and overall runtime (1.64s). In comparison, SR [13] requires 1.42s GPU time and 2.05s overall runtime, NSM [12] takes 1.68s / 2.57s, and DPR [61] + Reranker [62] requires 0.98s / 1.94s. The superior performance of M-ER can be attributed to its search strategy. SR [13] selects subgraphs based on nodes with the highest similarity scores, which often leads to redundant expansions on large-scale knowledge graphs. In contrast, M-ER avoids unnecessary exploration while retaining question-relevant nodes. This strategy effectively reduces computational overhead and improves answer accuracy.

In the second set of experiments (LLM-based Setting), we evaluate the overall runtime, input/output token counts, cost-effectiveness, and Hits@1 by comparing our method with other approaches, as shown in Table VI. The results indicate that M-ER (w/ LLM) achieves the best overall performance across most metrics. Specifically, M-ER attains the highest Hits@1 (78.88%) while exhibiting the lowest overall runtime (6.08s). In terms of token usage, M-ER consumes 706 input tokens and 320 output tokens per question, which is comparable to ToG [56] (679/326 tokens) and GoG [57] (548/406 tokens). Despite a comparable token budget, M-ER achieves superior performance, as reflected by its higher Hits@1 and cost-effectiveness than other baselines. In contrast, MCTS-KBQA [22] shows lower accuracy (74.68%) and cost-effectiveness (7.49) while consuming more tokens, indicating inefficient utilization of LLM calls. The superior performance of M-ER can be attributed to its search strategy and guidance mechanism. Although MCTS-KBQA [22] also uses MCTS, it lacks explicit guidance for selecting subsequent search directions and does not fully exploit the structural information within subgraphs. As a result, it often terminates at intermediate states as final answers, leading to lower accuracy and unstable performance.

In the third set of experiments (Search Strategy Comparison Setting), we evaluate overall runtime, input/output token counts, cost-effectiveness, and Hits@1 by comparing M-ER with different search strategies, as shown in Table VII. The results indicate that M-ER achieves the best overall performance across all metrics compared with other baselines. Specifically, M-ER attains the highest Hits@1 while also exhibiting the lowest overall runtime, resulting in the highest cost-effectiveness score. In comparison, ReAct [63] achieves a Hits@1 score of 73.23% and 10.38s overall runtime, while BFS [65] and DFS [64] show slightly higher overall runtime and lower accuracy than our method. Regarding LLM token consumption, M-ER requires fewer tokens than ReAct [63] (982/735), BFS [65] (904/382), and DFS [64] (857/356). Overall, these results demonstrate that integrating MCTS with LLMs in M-ER not only improves answer accuracy but also ensures more efficient utilization of computational resources.

Summary The cost-effectiveness analysis demonstrates that M-ER consistently outperforms other methods across all three experimental setups, achieving a lower overall runtime. These results further confirm the efficiency of our method.

TABLE V
THE WALL-CLOCK GPU HOURS, OVERALL RUNTIME, COST-EFFECTIVENESS, AND HITS@1 OF M-ER UNDER THE NON-LLM SETTING ON THE WEBQSP DATASET. **BOLD** NUMBERS DENOTE THE BEST PERFORMANCE, AND UNDERLINED NUMBERS DENOTE THE SECOND-BEST PERFORMANCE.

Method	Hits@1	Wall-clock GPU Time	Overall Runtime	Cost-Effectiveness
SR [13]	71.83	1.42	2.05	35.04
NSM [12]	67.18	1.68	2.57	26.14
DPR [61] + Reranker [62]	65.76	0.98	1.94	33.90
M-ER (w/o LLM)	**72.37**	**0.74**	**1.64**	**44.13**

TABLE VI
THE OVERALL RUNTIME, INPUT/OUTPUT TOKEN COUNTS, COST-EFFECTIVENESS, AND HITS@1 OF M-ER UNDER THE LLM-BASED SETTING ON THE WEBQSP DATASET. **BOLD** NUMBERS DENOTE THE BEST PERFORMANCE, AND UNDERLINED NUMBERS DENOTE THE SECOND-BEST PERFORMANCE.

Method	Hits@1	Overall Runtime	Input Tokens / Output Tokens	Cost-Effectiveness
ToG [56]	77.26	9.33	679/326	8.28
GoG [57]	76.24	9.79	**548**/406	7.79
MCTS-KBQA [22]	74.68	9.97	892/629	7.49
M-ER (w/ LLM)	**78.88**	**6.08**	706/**320**	**12.97**

TABLE VII
THE OVERALL RUNTIME, INPUT/OUTPUT TOKEN COUNTS, COST-EFFECTIVENESS, AND HITS@1 OF M-ER UNDER THE SEARCH STRATEGY COMPARISON SETTING ON THE WEBQSP DATASET. **BOLD** NUMBERS DENOTE THE BEST PERFORMANCE, AND UNDERLINED NUMBERS DENOTE THE SECOND-BEST PERFORMANCE.

Method	Hits@1	Overall Runtime	Input Tokens / Output Tokens	Cost-Effectiveness
ReAct [63]	73.23	10.38	982/735	7.05
BFS [65]	74.17	13.15	904/382	5.64
DFS [64]	75.24	13.06	857/356	5.76
M-ER	**78.88**	**6.08**	**706/320**	**12.97**

E. Ablation Study

To evaluate the impact of individual components in M-ER, we conduct ablation studies under the following settings:

- W/o Reflection: The reflection of the LLM is removed, and the subgraph is directly extended based on the similarity between the question and nodes.
- W/o Evaluation: The evaluation step is omitted, resulting in random node selection during each expansion step.
- W/o MCTS: Without MCTS, node search is based on similarity, consistent with SR [13].
- W/o LLM: This setting removes both the reflection and evaluation mechanisms of the LLM.

As can be seen from Table VIII, removing the LLM module has the greatest impact on model performance, with a 5.37% decrease in Hits@1. This indicates that the reflection and evaluation mechanisms of the LLM play a crucial role in improving overall performance. Specifically, the reflection mechanism directs the next search nodes based on the current subgraph, while the evaluation mechanism scores nodes in the current subgraph and feeds them back to the node selection stage. The removal of the reflection mechanism reduces Hits@1 by 1.86%, indicating that the mechanism also plays a significant role in guiding the subsequent search process. The removal of the MCTS module leads to a 4.59% performance degradation, indicating that MCTS effectively reduces redundant exploration by constraining the topological paths between topic entities and answer entities in the knowledge base. Finally, removing the evaluation module reduces Hits@1 by 1.2%.

TABLE VIII
ABLATION STUDY ON THE WEBQSP DATASET. THE BEST PERFORMANCE IN EACH COLUMN IS INDICATED IN **BOLD**, WHILE THE SECOND-BEST PERFORMANCE IS UNDERLINED.

Model	Hits@1	Hits@10	F1-score
W/o Reflection	77.02	81.64	67.24
W/o Evaluation	77.68	83.85	69.96
W/o MCTS	74.29	79.03	66.88
W/o LLM	73.51	77.21	64.97
Our method	**78.88**	**85.21**	**72.17**

By scoring the generated nodes at each step, the evaluation mechanism more accurately captures semantic dependencies relevant to the question.

Summary The ablation study demonstrates that integrating these modules reduces unnecessary reasoning steps, resulting in better overall performance compared to the ablated configurations.

F. Performance on Different LLMs

To investigate the impact of different LLMs, we compare the performance of six models on the WebQSP dataset, including llama3.1-8B, Qwen2-7B, DeepSeek-R1-Distill-Qwen-32B, GPT-3.5, GPT-4o, and DeepSeek-V3, as shown in Table IX. GPT-4o achieves the best Hits@1 score of 80.96% and ranks second on Hits@10 (84.49%) and F1-score (71.32). DeepSeek-V3 attains the highest Hits@10 (85.21%) and F1-score (72.17), while achieving a competitive Hits@1 of 78.88%. GPT-3.5 ranks second in Hits@1 (79.52%). DeepSeek-R1-Distill-Qwen-32B achieves 72.95% Hits@1, 74.08% Hits@10, and 67.69 F1-score, outperforming the smaller open-source models but remaining below GPT-3.5, GPT-4o, and DeepSeek-V3. In contrast, Qwen2-7B and llama3.1-8B exhibit lower overall performance. Specifically, Qwen2-7B achieves 70.68% Hits@1, 71.98% Hits@10, and 65.02 F1-score, while llama3.1-8B obtains 68.23% Hits@1, 69.21% Hits@10, and 64.03 F1-score. Overall, larger and more advanced models (e.g., GPT-4o and DeepSeek-V3) demonstrate clear advantages across multiple metrics, indicating stronger multi-hop reasoning capabilities. In contrast, smaller models show consistent performance degradation across all metrics, suggesting limitations in complex reasoning and semantic consistency.

Summary The comparative analysis highlights performance variations across different models. While smaller models, such as llama3.1-8B, achieve reasonable performance, DeepSeek-V3 demonstrates an excellent ability to align semantic dependencies.

TABLE IX
PERFORMANCE ON DIFFERENT LLMS ON THE WEBQSP DATASET. THE BEST PERFORMANCE IN EACH COLUMN IS INDICATED IN **BOLD**, WHILE THE SECOND-BEST PERFORMANCE IS UNDERLINED.

Model	Hits@1	Hits@10	F1-score
llama3.1-8B	68.23	69.21	64.03
Qwen2-7B	70.68	71.98	65.02
DeepSeek-R1-Distill-Qwen-32B	72.95	74.08	67.69
GPT-3.5	79.52	81.48	68.65
GPT-4o	**80.96**	84.49	71.32
DeepSeek-V3	78.88	**85.21**	**72.17**

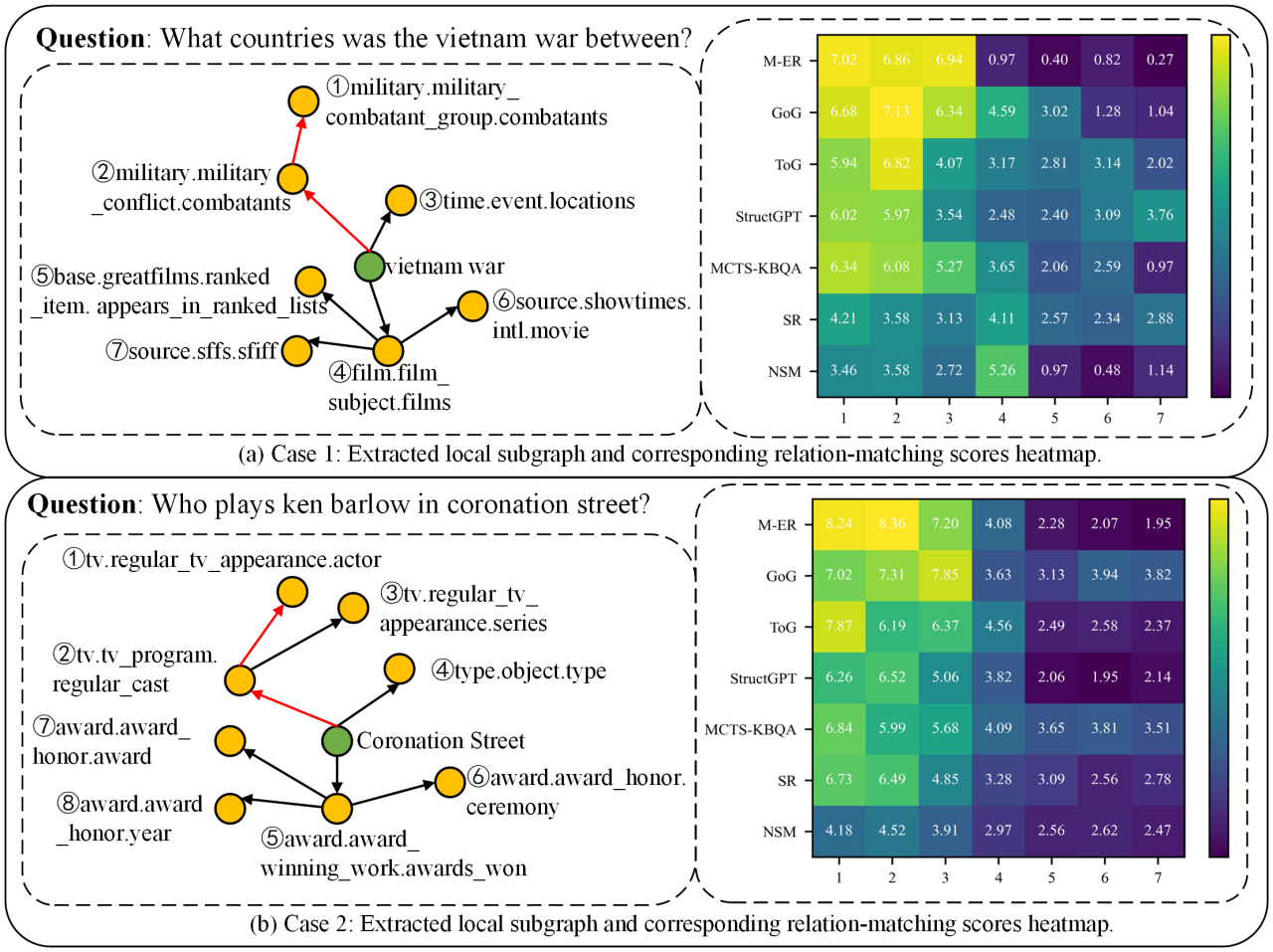


Fig. 6. Illustration of the extracted local subgraph and the corresponding relation-matching score heatmap, where the numbers denote relation indices. The red arrows indicate the correct reasoning path.

G. Case Study

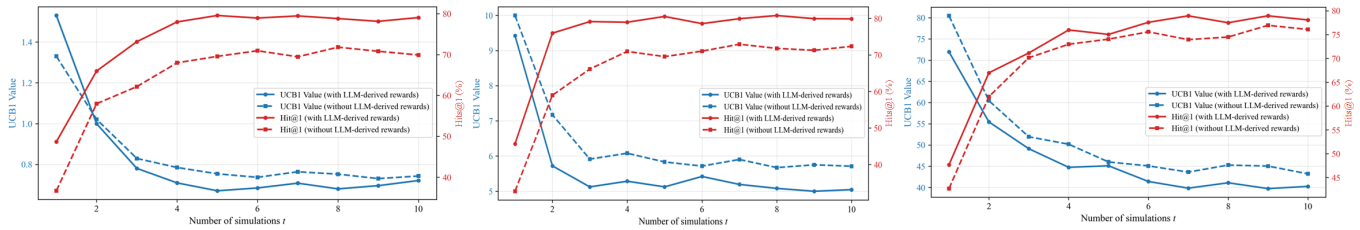
Subgraphs, as effective path combinations between entities, provide more focused insight than a complete KB by revealing the reasoning behind each target link. To further illustrate the advantages of our method, we conduct two case studies on the WebQSP dataset to evaluate whether it assigns higher relation–question similarity scores and better captures relevant contextual information, as shown in Figure 6. For each case, we present the extracted local subgraph and report the relation-matching scores—computed by our method and several representative baselines—between the question and each candidate relation.

Across both cases in Figure 6, M-ER exhibits a clear separation between contextually relevant and irrelevant relations in its similarity scores. This pattern suggests that M-ER captures the semantic dependencies between the question and candidate relations more accurately than existing baseline methods. Case 2 illustrates this contrast most clearly. M-ER produces sharp peaks for the correct relations and yields the largest gap between high and low similarity scores. In particular, it assigns high scores to the role-related relations (relations 1-3), with relation 1 reaching a score of 8.24, while assigning low scores to the award-related relations (relations 5–8), with relation 8 decreasing to 1.95. In comparison, StructGPT [58] and MCTS-KBQA [22] exhibit much flatter score distributions, which makes it difficult to distinguish relevant relations from irrelevant ones. NSM [12] shows an even smaller margin, with only a difference of 2.05 between its highest and lowest scores. Case 1 demonstrates a similar overall trend. Overall, these two cases demonstrate that M-ER yields more discriminative similarity patterns and clearer semantic separation between relevant and irrelevant relations. This enhanced discrimination facilitates more accurate identification of relevant relations, resulting in higher-quality subgraphs and better question-answering performance.

Summary These case studies demonstrate that our method improves semantic discrimination in subgraph extraction. Compared to SR and other baselines, such as GoG, ToG, StructGPT, and MCTS-KBQA, our approach more effectively identifies contextually relevant nodes while suppressing irrelevant ones, thus enhancing the quality of the subgraph.

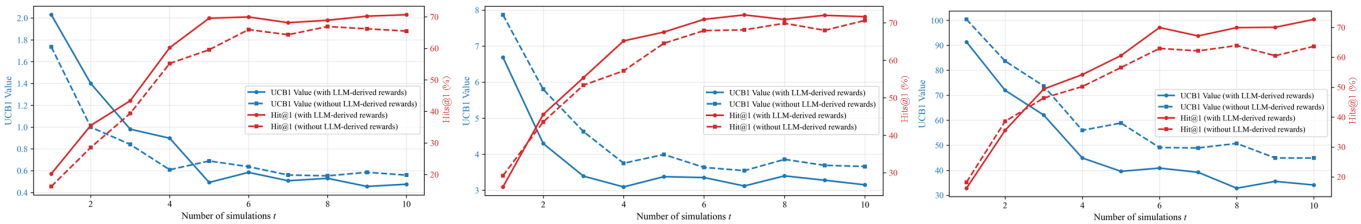
H. Discussion

1) Empirical Analysis of UCB1 Convergence: To analyze the empirical convergence of UCB1 [41] and the iteration count required for convergence, we evaluate both the con-



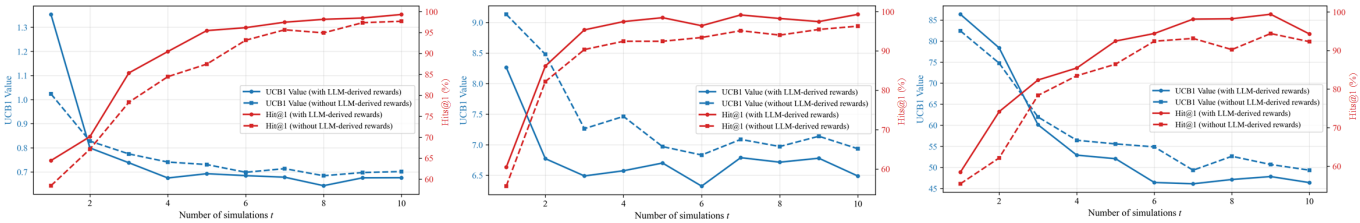
(a) Interval [0, 1] (b) Interval [0, 10] (c) Interval [0, 100]

Fig. 7. Empirical convergence of UCB1 and Hits@1 over 10 MCTS iterations under different reward ranges on the WebQSP dataset for two methods (with LLM-derived rewards and without LLM-derived rewards). The (a)–(c) correspond to the results for reward ranges of [0, 1], [0, 10], and [0, 100], respectively. The UCB1 curve (blue) starts from its upper bound and decreases toward the stable value. Simultaneously, Hits@1 (red) curve plateaus.



(a) Interval [0, 1] (b) Interval [0, 10] (c) Interval [0, 100]

Fig. 8. Empirical convergence of UCB1 and Hits@1 over 10 MCTS iterations under different reward intervals on the CWQ dataset for two methods (with LLM-derived rewards and without LLM-derived rewards). The (a)–(c) correspond to the results for reward ranges of [0, 1], [0, 10], and [0, 100], respectively. The UCB1 curve (blue) starts from its upper bound and decreases toward the stable value. Simultaneously, Hits@1 (red) curve plateaus.



(a) Interval [0, 1] (b) Interval [0, 10] (c) Interval [0, 100]

Fig. 9. Empirical convergence of UCB1 and Hits@1 over 10 MCTS iterations under different reward intervals on the MetaQA-3hop dataset for two methods (with LLM-derived rewards and without LLM-derived rewards). The (a)–(c) correspond to the results for reward ranges of [0, 1], [0, 10], and [0, 100], respectively. The UCB1 curve (blue) starts from its upper bound and decreases toward the stable value. Simultaneously, Hits@1 (red) curve plateaus.

vergence curves of UCB1 and Hits@1 on three benchmark datasets (WebQSP, CWQ, and MetaQA-3hop datasets). The experiments are conducted under two conditions, with LLM-derived rewards and without LLM-derived rewards, to analyze the impact of LLM-derived rewards on convergence. Specifically, the range of LLM-derived rewards is treated as a hyperparameter, since its scale directly affects the exploration–exploitation balance in UCB1, thereby influencing convergence speed and overall performance. Following the configurations adopted in prior work, we evaluate three representative reward ranges, including [0, 1] [17], [0, 10] [25], [66], and [0, 100] [2], [67]. To ensure a fair comparison across different methods, all other parameters are kept unchanged [17], [42]. For each reward range ([0,1], [0,10], [0,100]), we use the same set of 5 randomly sampled points. In each iteration, the UCB1 values of these points are averaged to produce the UCB1 value for that iteration. This process is repeated for 10 iterations to generate the UCB1 convergence curve for each reward range. The corresponding results are reported in Figures 7-9. These results demonstrate that incorporating LLM-derived rewards accelerates convergence and improves performance compared with the setting without LLM-derived rewards. Among the tested ranges, the [0, 10] reward range achieves faster convergence and higher final performance than other ranges. Specifically, the performance of the method with LLM-derived rewards converges after approximately 3 iterations on both the WebQSP and MetaQA datasets, and after 5 iterations on the CWQ dataset. These findings indicate that the integration of LLM-derived rewards does not violate the boundedness or stationarity assumptions required by the convergence of UCB1. The number of iterations required to achieve converged performance can be set as a reliable computational budget for practical applications.

2) Big-O Analysis: To assess the practical applicability of our method, we provide a Big-O analysis of our method and compare its complexity with both MCTS-based methods and other search strategies.

Specifically, we follow the approach used in KBQA-o1

[21] to assess the search space complexity of our method. Let k denote the branching factor and d the search depth. Our method operates through four stages: Selection, Expansion, Simulation, and Backpropagation. **Selection Stage:** The algorithm traverses the search tree up to depth d, selecting the best node using UCB1 [41], resulting in a complexity of $O(k \cdot d)$ per rollout. **Expansion Stage:** The B candidates are generated from k branches, introducing an additional complexity of $O(\omega k)$, where ω denotes the average search space complexity required for LLM evaluation and reflection, as well as similarity-matching operations, on each branch. **Simulation Stage:** This stage explores paths up to depth d, contributing $O(k \cdot d)$ complexity. **Backpropagation Stage:** The rewards are updated along the path, requiring $O(d)$. The total search space complexity for a single rollout is $O(k \cdot d)$. Therefore, with N rollouts, the overall search space complexity of our method is $O(N \cdot k \cdot d)$, which scales linearly with both the number of rollouts and the search depth. Compared with existing MCTS-based KBQA methods such as KBQA-o1 [21] and MCTS-KBQA [22], our method has the same search space. In contrast to other search strategies like DFS [64] and BFS [65], which have exponential search complexity $O(k^d)$, our method focuses on promising nodes, reducing unnecessary computation and maintaining efficiency.

V. Conclusion

This paper proposes the M-ER framework to address multi-hop KBQA challenges through synergistic integration of MCTS and LLM. Our approach overcomes the limitations of conventional search strategies by establishing a dynamic reasoning loop that combines semantic guidance with the selection of relevant nodes for the question. Experimental results on the WebQSP dataset demonstrate the M-ER's effectiveness in suppressing noisy relations while maintaining 78.88% Hits@1.

References

[1] C. Unger, A. Freitas, and P. Cimiano, "An introduction to question answering over linked data," *Reasoning Web. Reasoning on the Web in the Big Data Era: 10th International Summer School 2014, Athens, Greece, September 8-13, 2014. Proceedings 10*, pp. 100–140, 2014.

[2] K. Bollacker, C. Evans, P. Paritosh, T. Sturge, and J. Taylor, "Freebase: a collaboratively created graph database for structuring human knowledge," in *Proceedings of the 2008 ACM SIGMOD international conference on Management of data*, 2008, pp. 1247–1250.

[3] J. Lehmann, R. Isele, M. Jakob, A. Jentzsch, D. Kontokostas, P. N. Mendes, S. Hellmann, M. Morsey, P. Van Kleef, S. Auer *et al.*, "Dbpedia–a large-scale, multilingual knowledge base extracted from wikipedia," *Semantic web*, vol. 6, no. 2, pp. 167–195, 2015.

[4] Y. Lan, S. Wang, and J. Jiang, "Knowledge base question answering with topic units," 2019.

[5] H. Sun, B.-W. Tania, and C. William, "Pullnet: Open domain question answering with iterative retrieval on knowledge bases and text," in *Proceedings of the 2019 Conference on Empirical Methods in Natural Language Processing and the 9th International Joint Conference on Natural Language Processing (EMNLP-IJCNLP)*, 2019, pp. 2380–2390.

[6] Y. Sun, L. Zhang, G. Cheng, and Y. Qu, "Sparqa: skeleton-based semantic parsing for complex questions over knowledge bases," in *Proceedings of the AAAI conference on artificial intelligence*, vol. 34, no. 05, 2020, pp. 8952–8959.

[7] S. Ravishankar, D. Thai, I. Abdelaziz, N. Mihindukulasooriya, T. Naseem, P. Kapanipathi, G. Rossiello, and A. Fokoue-Nkoutche, "A two-stage approach towards generalization in knowledge base question answering," in *Findings of the Association for Computational Linguistics: EMNLP 2022*, 2022, pp. 5571–5580.

[8] Y. Lan and J. Jiang, "Query graph generation for answering multi-hop complex questions from knowledge bases." Association for Computational Linguistics, 2020.

[9] R. Das, M. Zaheer, D. Thai, A. Godbole, E. Perez, J.-Y. Lee, L. Tan, L. Polymenakos, and A. McCallum, "Case-based reasoning for natural language queries over knowledge bases," *arXiv preprint arXiv:2104.08762*, 2021.

[10] G. Xiong, J. Bao, and W. Zhao, "Interactive-kbqa: Multi-turn interactions for knowledge base question answering with large language models," in *Proceedings of the 62nd Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers)*, 2024, pp. 10 561–10 582.

[11] D. Luo, J. Sheng, H. Xu, L. Wang, and B. Wang, "Improving complex knowledge base question answering with relation-aware subgraph retrieval and reasoning network," in *2023 International Joint Conference on Neural Networks (IJCNN)*. IEEE, 2023, pp. 1–8.

[12] G. He, Y. Lan, J. Jiang, W. X. Zhao, and J.-R. Wen, "Improving multi-hop knowledge base question answering by learning intermediate supervision signals," in *Proceedings of the 14th ACM international conference on web search and data mining*, 2021, pp. 553–561.

[13] J. Zhang, X. Zhang, J. Yu, J. Tang, J. Tang, C. Li, and H. Chen, "Subgraph retrieval enhanced model for multi-hop knowledge base question answering," *arXiv preprint arXiv:2202.13296*, 2022.

[14] J. Dong, Q. Zhang, C. Zhou, H. Chen, D. Zha, and X. Huang, "Cost-efficient knowledge-based question answering with large language models," *arXiv preprint arXiv:2405.17337*, 2024.

[15] Z. Zhao, W. S. Lee, and D. Hsu, "Large language models as commonsense knowledge for large-scale task planning," *Advances in Neural Information Processing Systems*, vol. 36, pp. 31 967–31 987, 2023.

[16] Z. Qi, M. Ma, J. Xu, L. L. Zhang, F. Yang, and M. Yang, "Mutual reasoning makes smaller llms stronger problem-solvers," *arXiv preprint arXiv:2408.06195*, 2024.

[17] H. Tran, Z. Yao, J. Wang, Y. Zhang, Z. Yang, and H. Yu, "Rare: Retrieval-augmented reasoning enhancement for large language models," *arXiv preprint arXiv:2412.02830*, 2024.

[18] S. Li, S. Dong, K. Luan, X. Di, and C. Ding, "Enhancing reasoning through process supervision with monte carlo tree search," *arXiv preprint arXiv:2501.01478*, 2025.

[19] W. Feng, C. Hao, Y. Zhang, J. Song, and H. Wang, "Airrag: Activating intrinsic reasoning for retrieval augmented generation via tree-based search," *arXiv preprint arXiv:2501.10053*, 2025.

[20] J. Jiang, Z. Chen, Y. Min, J. Chen, X. Cheng, J. Wang, Y. Tang, H. Sun, J. Deng, W. X. Zhao *et al.*, "Technical report: Enhancing llm reasoning with reward-guided tree search," *arXiv preprint arXiv:2411.11694*, 2024.

[21] H. Luo, Y. Guo, Q. Lin, X. Wu, X. Mu, W. Liu, M. Song, Y. Zhu, L. A. Tuan *et al.*, "Kbqa-o1: Agentic knowledge base question answering with monte carlo tree search," *arXiv preprint arXiv:2501.18922*, 2025.

[22] G. Xiong, H. Li, and W. Zhao, "Mcts-kbqa: Monte carlo tree search for knowledge base question answering," *arXiv preprint arXiv:2502.13428*, 2025.

[23] K. Liang, L. Meng, H. Li, J. Wang, L. Lan, M. Li, X. Liu, and H. Wang, "From concrete to abstract: multi-view clustering on relational knowledge," *IEEE Transactions on Pattern Analysis and Machine Intelligence*, 2025.

[24] K. Liang, L. Meng, M. Liu, Y. Liu, W. Tu, S. Wang, S. Zhou, and X. Liu, "Learn from relational correlations and periodic events for temporal knowledge graph reasoning," in *Proceedings of the 46th international ACM SIGIR conference on research and development in information retrieval*, 2023, pp. 1559–1568.

[25] B. Xu, Y. Lin, Y. Li, and Y. Gao, "Sra-mcts: Self-driven reasoning augmentation with monte carlo tree search for code generation," in *Proceedings of the Thirty-Fourth International Joint Conference on Artificial Intelligence, IJCAI-25*, J. Kwok, Ed. International Joint Conferences on Artificial Intelligence Organization, 8 2025, pp. 8678–8686, main Track. [Online]. Available: https://doi.org/10.24963/ijcai.2025/965

[26] A. Miller, A. Fisch, J. Dodge, A.-H. Karimi, A. Bordes, and J. Weston, "Key-value memory networks for directly reading documents," in *Proceedings of the 2016 Conference on Empirical Methods in Natural Language Processing*. Association for Computational Linguistics, 2016.

[27] H. Sun, B. Dhingra, M. Zaheer, K. Mazaitis, R. Salakhutdinov, and W. Cohen, "Open domain question answering using early fusion of knowledge bases and text," in *Proceedings of the 2018 Conference on Empirical Methods in Natural Language Processing*, E. Riloff, D. Chiang, J. Hockenmaier, and J. Tsujii, Eds. Brussels, Belgium: Association for Computational Linguistics, Oct.-Nov. 2018, pp. 4231–4242. [Online]. Available: https://aclanthology.org/D18-1455/

[28] L. Zhang, J. Zhang, Y. Wang, S. Cao, X. Huang, C. Li, H. Chen, and J. Li, "FC-KBQA: A fine-to-coarse composition framework for knowledge base question answering," in *Proceedings of the 61st Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers)*, A. Rogers, J. Boyd-Graber, and N. Okazaki, Eds. Toronto, Canada: Association for Computational Linguistics, Jul. 2023, pp. 1002–1017. [Online]. Available: https://aclanthology.org/2023.acl-long.57/

[29] R. Das, A. Godbole, A. Naik, E. Tower, R. Jia, M. Zaheer, H. Hajishirzi, and A. McCallum, "Knowledge base question answering by case-based reasoning over subgraphs," 2022. [Online]. Available: https://arxiv.org/abs/2202.10610

[30] Y. Jia and W. Chen, "A method of query graph reranking for knowledge base question answering," *arXiv preprint arXiv:2204.12808*, 2022.

[31] X. Chen, S. Zhou, K. Liang, and X. Liu, "Distilling reasoning ability from large language models with adaptive thinking," *IEEE Transactions on Neural Networks and Learning Systems*, vol. 36, no. 11, pp. 19 820–19 833, 2025.

[32] X. Chen, S. Zhou, K. Liang, D. Yuan, H. Chen, X. Sun, L. Meng, and X. Liu, "Putting on the thinking hats: A survey on chain of thought fine-tuning from the perspective of human reasoning mechanism," *arXiv preprint arXiv:2510.13170*, 2025.

[33] H. Luo, E. Haihong, Z. Tang, S. Peng, Y. Guo, W. Zhang, C. Ma, G. Dong, M. Song, W. Lin *et al.*, "Chatkbqa: A generate-then-retrieve framework for knowledge base question answering with fine-tuned large language models," in *Findings of the Association for Computational Linguistics ACL 2024*, 2024, pp. 2039–2056.

[34] M. Patidar, R. Sawhney, A. Singh, B. Chatterjee, I. Bhattacharya *et al.*, "Few-shot transfer learning for knowledge base question answering: Fusing supervised models with in-context learning," in *Proceedings of the 62nd Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers)*, 2024, pp. 9147–9165.

[35] Z. Shi, S. Zhang, W. Sun, S. Gao, P. Ren, Z. Chen, and Z. Ren, "Generate-then-ground in retrieval-augmented generation for multi-hop question answering," in *Proceedings of the 62nd Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers)*, 2024, pp. 7339–7353.

[36] L. Luo, Y.-F. Li, G. Haffari, and S. Pan, "Reasoning on graphs: Faithful and interpretable large language model reasoning," *arXiv preprint arXiv:2310.01061*, 2023.

[37] Y. Liu, Z. Li, X. Jin, Y. Guo, L. Bai, S. Guan, J. Guo, and X. Cheng, "An in-context schema understanding method for knowledge base question answering," in *International Conference on Knowledge Science, Engineering and Management*. Springer, 2024, pp. 419–434.

[38] Y. Tian, P. Yang, D. Song, Z. Wu, and H. Wang, "Grv-kbqa: A three-stage framework for knowledge base question answering with decoupled logical structure, semantic grounding and structure-aware validation," in *Findings of the Association for Computational Linguistics: EMNLP 2025*, 2025, pp. 2618–2632.

[39] T. Feng and L. He, "Rgr-kbqa: Generating logical forms for question answering using knowledge-graph-enhanced large language model," in *Proceedings of the 31st International Conference on Computational Linguistics*, 2025, pp. 3057–3070.

[40] X. Wang, S. Zhao, B. Cheng, J. Han, L. Yingting, H. Yang, I. Sekulic, and G. Nan, "Integrating subgraph-aware relation and direction reasoning for question answering," in *ICASSP 2021-2021 IEEE International Conference on Acoustics, Speech and Signal Processing (ICASSP)*. IEEE, 2021, pp. 7808–7812.

[41] P. Auer, N. Cesa-Bianchi, and P. Fischer, "Finite-time analysis of the multiarmed bandit problem," *Machine learning*, vol. 47, pp. 235–256, 2002.

[42] Y. Lin, X. Ouyang, T. Zhang, and K. Sui, "Rpm-mcts: Knowledge-retrieval as process reward model with monte carlo tree search for code generation," *arXiv preprint arXiv:2511.19895*, 2025.

[43] D. Zhang, S. Zhoubian, Z. Hu, Y. Yue, Y. Dong, and J. Tang, "Rest-mcts*: Llm self-training via process reward guided tree search," *Advances in Neural Information Processing Systems*, vol. 37, pp. 64 735–64 772, 2024.

[44] W.-t. Yih, M. Richardson, C. Meek, M.-W. Chang, and J. Suh, "The value of semantic parse labeling for knowledge base question answering," in *Proceedings of the 54th Annual Meeting of the Association for Computational Linguistics (Volume 2: Short Papers)*, 2016, pp. 201–206.

[45] Y. Wang, D. Lian, and E. Chen, "Event assignment based on kbqa for government service hotlines," *Applied Artificial Intelligence*, vol. 38, no. 1, p. 2348162, 2024.

[46] A. Talmor and J. Berant, "The web as a knowledge-base for answering complex questions," *arXiv preprint arXiv:1803.06643*, 2018.

[47] Y. Zhang, H. Dai, Z. Kozareva, A. Smola, and L. Song, "Variational reasoning for question answering with knowledge graph," in *Proceedings of the AAAI conference on artificial intelligence*, vol. 32, no. 1, 2018.

[48] Z. Liu, C. Wu, N. Shao, S. Xiao, C. Li, and D. Lian, "Lighter and better: Towards flexible context adaptation for retrieval augmented generation," *arXiv preprint arXiv:2409.15699*, 2024.

[49] DeepSeek-AI, "Deepseek-v3 technical report," 2024. [Online]. Available: https://arxiv.org/abs/2412.19437

[50] S. Sharma, S. Tuli, and N. Badam, "Challenges and applications of large language models: A comparison of gpt and deepseek family of models," *arXiv preprint arXiv:2508.21377*, 2025.

[51] A. Saxena, A. Tripathi, and P. Talukdar, "Improving multi-hop question answering over knowledge graphs using knowledge base embeddings," in *Proceedings of the 58th annual meeting of the association for computational linguistics*, 2020, pp. 4498–4507.

[52] N. Bhutani, X. Zheng, and H. V. Jagadish, "Learning to answer complex questions over knowledge bases with query composition," in *Proceedings of the 28th ACM International Conference on Information and Knowledge Management*, ser. CIKM '19. New York, NY, USA: Association for Computing Machinery, 2019, p. 739–748. [Online]. Available: https://doi.org/10.1145/3357384.3358033

[53] N. Bhutani, X. Zheng, K. Qian, Y. Li, and H. Jagadish, "Answering complex questions by combining information from curated and extracted knowledge bases," in *Proceedings of the first workshop on natural language interfaces*, 2020, pp. 1–10.

[54] L. Ma, P. Zhang, X. Zhu, D. Luo, and B. Wang, "Structure-aware reasoning for knowledge base question answering," in *Pacific-Asia Conference on Knowledge Discovery and Data Mining*. Springer, 2022, pp. 562–573.

[55] W. Jin, B. Zhao, H. Yu, X. Tao, R. Yin, and G. Liu, "Improving embedded knowledge graph multi-hop question answering by introducing relational chain reasoning," *Data Mining and Knowledge Discovery*, vol. 37, no. 1, pp. 255–288, 2023.

[56] J. Sun, C. Xu, L. Tang, S. Wang, C. Lin, Y. Gong, L. M. Ni, H.-Y. Shum, and J. Guo, "Think-on-graph: Deep and responsible reasoning of large language model on knowledge graph," *arXiv preprint arXiv:2307.07697*, 2023.

[57] Y. Xu, S. He, J. Chen, Z. Wang, Y. Song, H. Tong, G. Liu, J. Zhao, and K. Liu, "Generate-on-graph: Treat llm as both agent and kg for incomplete knowledge graph question answering," in *Proceedings of the 2024 Conference on Empirical Methods in Natural Language Processing*, 2024, pp. 18 410–18 430.

[58] J. Jiang, K. Zhou, Z. Dong, K. Ye, X. Zhao, and J.-R. Wen, "StructGPT: A general framework for large language model to reason over structured data," in *Proceedings of the 2023 Conference on Empirical Methods in Natural Language Processing*, H. Bouamor, J. Pino, and K. Bali, Eds. Singapore: Association for Computational Linguistics, Dec. 2023, pp. 9237–9251. [Online]. Available: https://aclanthology.org/2023.emnlp-main.574/

[59] H. W. Chung, L. Hou, S. Longpre, B. Zoph, Y. Tai, W. Fedus, Y. Li, X. Wang, M. Dehghani, S. Brahma, A. Webson, S. S. Gu, Z. Dai, M. Suzgun, X. Chen, A. Chowdhery, A. Castro-Ros, M. Pellat, K. Robinson, D. Valter, S. Narang, G. Mishra, A. Yu, V. Zhao, Y. Huang, A. Dai, H. Yu, S. Petrov, E. H. Chi, J. Dean, J. Devlin, A. Roberts, D. Zhou, Q. V. Le, and J. Wei, "Scaling instruction-finetuned language models," *J. Mach. Learn. Res.*, vol. 25, no. 1, Mar. 2025.

[60] Z. Li, S. Fan, Y. Gu, X. Li, Z. Duan, B. Dong, N. Liu, and J. Wang, "Flexkbqa: A flexible llm-powered framework for few-shot knowledge base question answering," in *Proceedings of the AAAI conference on artificial intelligence*, vol. 38, no. 17, 2024, pp. 18 608–18 616.

[61] V. Karpukhin, B. Oğuz, S. Min, P. Lewis, L. Wu, S. Edunov, D. Chen, and W. T. Yih, "Dense passage retrieval for open-domain question answering," in *2020 Conference on Empirical Methods in Natural Language Processing, EMNLP 2020*. Association for Computational Linguistics (ACL), 2020, pp. 6769–6781.

[62] S. Xiao, Z. Liu, P. Zhang, N. Muennighoff, D. Lian, and J.-Y. Nie, "C-pack: Packed resources for general chinese embeddings," in *Proceedings of the 47th international ACM SIGIR conference on research and development in information retrieval*, 2024, pp. 641–649.

[63] S. Yao, J. Zhao, D. Yu, N. Du, I. Shafran, K. Narasimhan, and Y. Cao, "React: Synergizing reasoning and acting in language models," in *International Conference on Learning Representations (ICLR)*, 2023.

[64] V. Etemadi and G. Robles, "Smartdoc: A context-aware agentic method comment generation plugin," *arXiv preprint arXiv:2511.00450*, 2025.

[65] J. Jiang, K. Zhou, W. X. Zhao, Y. Li, and J.-R. Wen, "Reasoninglm: Enabling structural subgraph reasoning in pre-trained language models for question answering over knowledge graph," in *Proceedings of the*

2023 Conference on Empirical Methods in Natural Language Processing, 2023, pp. 3721–3735.

[66] L. Chang, "Magellan: Guided mcts for latent space exploration and novelty generation," *arXiv preprint arXiv:2510.21341*, 2025.

[67] G. Li, Y. Liu, Z. Qin, Y. Wang, J. Zhong, C. Zhi, B. Li, F. Huang, Y. Li, and S. Deng, "Empowering repoqa-agent based on reinforcement learning driven by monte-carlo tree search," *arXiv preprint arXiv:2510.26287*, 2025.

Duanyang Yuan is currently pursuing a Ph.D. degree at the National University of Defense Technology (NUDT). Her research interests include knowledge graphs, large language models, information retrieval, and retrieval-augmented generation. She has published several papers in AAAI, Frontiers in Physiology, etc.

Sihang Zhou received his PhD degree from School of Computer, National University of Defense Technology (NUDT), China. He is now an associate professor at the College of Intelligence Science and Technology, NUDT. His current research interests include machine learning, knowledge graph, and medical image analysis. Dr. Zhou has published 40+ peer-reviewed papers, including IEEE T-IP, IEEE T-NNLS, IEEE T-MI, Medical Image Analysis, AAAI, MICCAI, etc.

Xiaoshu Chen is currently pursuing a Ph.D. degree at the National University of Defense Technology (NUDT). Before joining NUDT, he got his MSc degree at WuHan University (WHU) and BSc degree at China University of Mining and Technology (CUMT). His research interests include large language models, knowledge graph, information retrieval and semantic segmentation. He has published several papers in AAAI, IEEE TGRS and Neurocomputing, etc.

Haoyuan Chen is currently pursuing a Ph.D. degree at the National University of Defense Technology (NUDT). His research interests include large language models, retrieval-augmented generation, and multi-agent systems.

Ke Liang received his Ph.D. degree at the National University of Defense Technology (NUDT). He got his BSc degree at Beihang University (BUAA) and received his MSc degree from the Pennsylvania State University (PSU). His current research interests include knowledge graphs, graph learning, and healthcare AI. He has published several papers in highly regarded journals and conferences such as SIGIR, AAAI, ICML, ACM MM, IEEE TNNLS, IEEE TKDE, etc.

Jian Huang received the Ph.D. degree from the College of Mechatronics and Automation, National University of Defense Technology, Changsha, China, in 2000. She is now a professor at the College of Intelligence Science, National University of Defense Technology, China. Her current research interests include system simulation and artificial intelligence.